

Esercizio di programmazione concorrente

Ci sono tanti utenti, ciascuno appartenente a uno di tre gruppi (A, B e C).

Ciascun utente è implementato come un thread che, ciclicamente

- 1) chiede accesso alla risorsa;
- 2) quando ha ottenuto l'accesso alla risorsa, la usa per un po';
- 3) rilascia la risorsa;
- 4) fa altre operazioni che non richiedono l'uso della risorsa.

Mentre un utente di un certo gruppo usa la risorsa, gli utenti degli altri gruppi non la possono usare. Viceversa, gli utenti dello stesso gruppo la possono usare, se ottengono l'accesso in base ai criteri descritti nel seguito.

I metodi di accesso alla risorsa fanno in modo che tutti i gruppi possano prima o poi accedere alla risorsa. A questo scopo

- La variabile tipoDiTurno indica quale tipo (A, B, C o nessuno dei tre) ha correntemente accesso alla risorsa.
- La struttura dati attesaUtenti registra quando un dato gruppo ha iniziato ad attendere di poter accedere alla risorsa. Questa struttura serve per poter assegnare la risorsa in modo equo: quando bisogna scegliere un gruppo a cui dare accesso, si sceglie quello in attesa da più tempo.
- Si contano gli accessi (numAccessi) e i rilasci (numRilasci) avvenuti da quando la risorsa è stata assegnata al tipo di turno.

L'assegnazione della risorsa deve evitare la starvation dei tipi; cioè si vuole che almeno un utente di ogni tipo prima o poi possa accedere alla risorsa, se lo desidera. NB: non interessa se uno specifico utente non riesce mai ad accedere alla risorsa, purché altri del suo stesso tipo accedano.

La risorsa viene dunque assegnata secondo i seguenti criteri:

1. Se nessuno la sta usando, il primo utente che arriva la può occupare, rendendola così accessibile agli altri utenti del suo gruppo;
2. Dopo TOT accessi da parte di utenti dello stesso tipo, la risorsa viene assegnata al tipo di utenti da più tempo in attesa. Se non c'è alcun tipo in attesa, la risorsa viene lasciata libera, e sarà occupata dal primo arrivato (vedi punto 1).
3. Quando non ci sono utenti che stanno usando la risorsa, questa viene lasciata libera, anche se non sono stati fatti TOT accessi, e sarà occupata dal primo arrivato (vedi punto 1).

Si richiede di modificare il codice dato in modo che il sistema si comporti come descritto sopra. Gli utenti che non possono accedere alla risorsa devono essere messi in attesa che si verifichino le condizioni per poter accedere (non è accettabile fare busy wait).

Il codice prodotto deve evitare deadlock e altri problemi tipici della programmazione concorrente. Per la starvation vale quanto detto sopra.

Esercizio di programmazione distribuita (con uso di Socket)

Si desidera un sistema distribuito che funziona come il sistema descritto nell'esercizio di programmazione concorrente, ma in cui la risorsa e gli utenti possono risiedere su macchine diverse. Il sistema deve essere realizzato usando socket per la comunicazione.

Esercizio di programmazione distribuita (con uso di RMI)

Si desidera un sistema distribuito che funziona come il sistema descritto nell'esercizio di programmazione concorrente, ma in cui la risorsa e gli utenti possono risiedere su macchine diverse. Il sistema deve essere realizzato usando RMI.